

# COMPUTER ARCHITECTURE

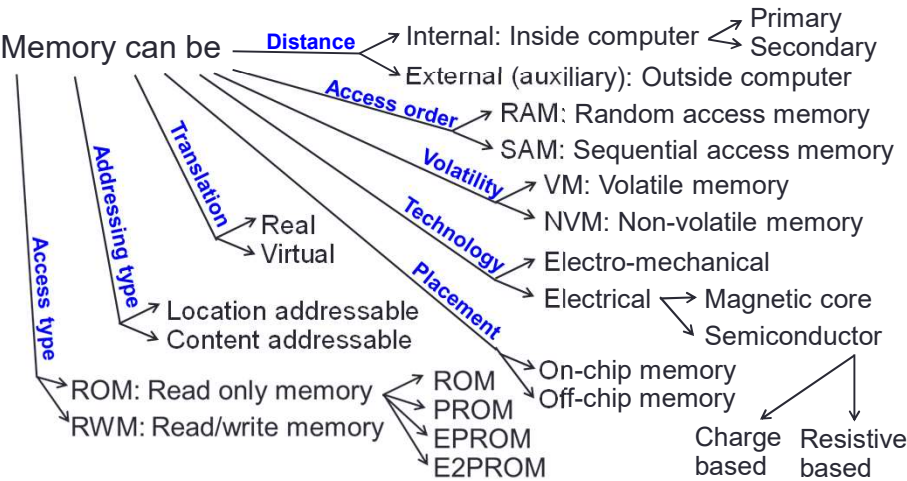


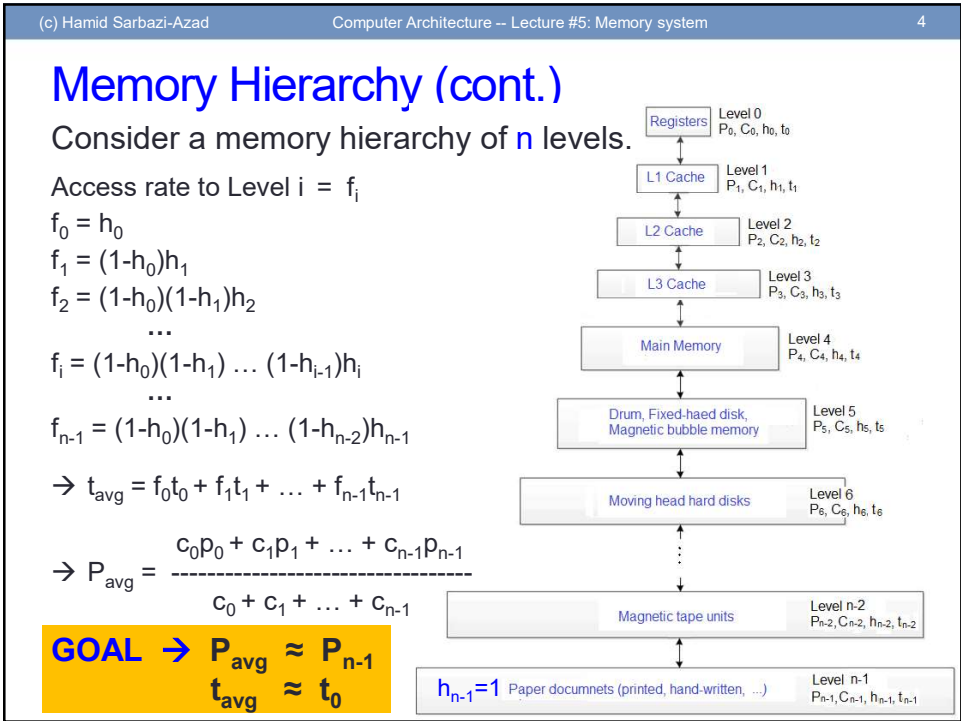
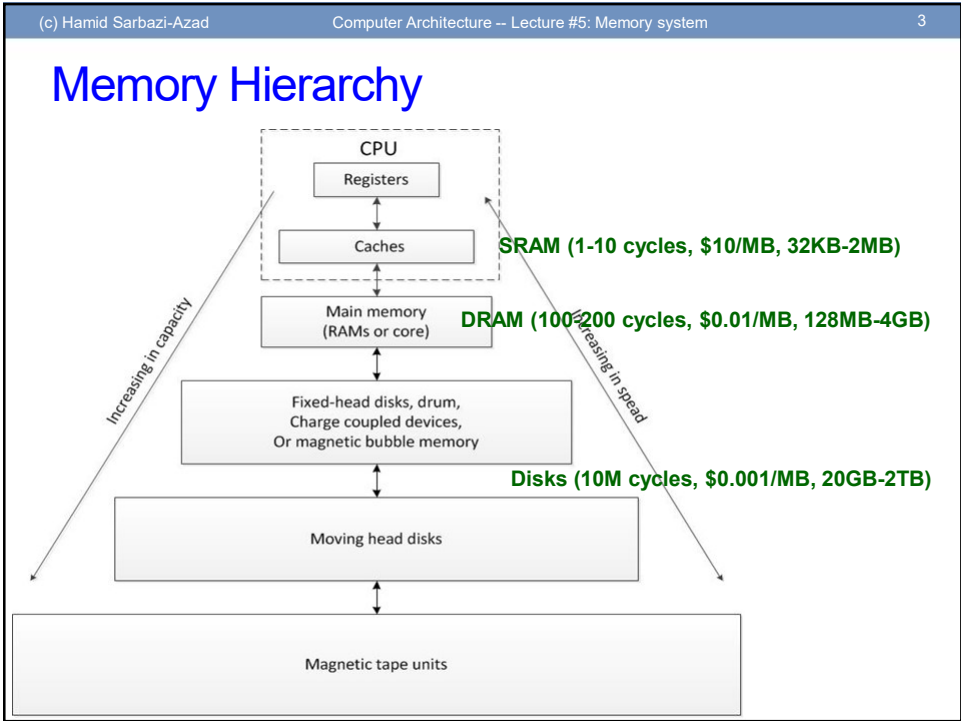
Hamid Sarbazi-Azad  
Department of Computer Engineering  
Sharif University of Technology  
Tehran, Iran

## Memory

Whatever that can store information which can be later read back when needed is called Memory!

**Example:** Stone, leather, papyrus, paper, disk, tape, ...





(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

5

# Read/write Memory (RWM aka RAM)

Block diagram of a  $2^n \times m$  bits RAM with its access lines.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

6

# Core Memory

Early memories were of magnetic core type.

Read is destructive (must write the read data after read).

→ Read latency = 2 x Write latency

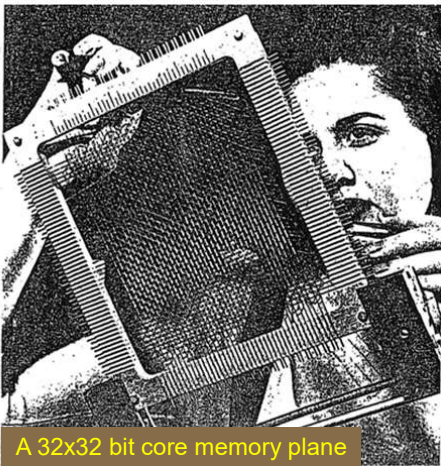
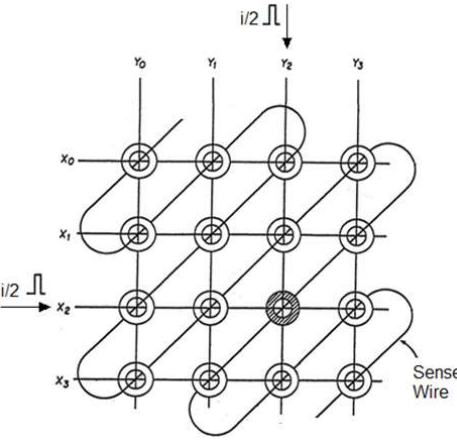
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

7

### Core Memory (2D arrangement)

To reduce wiring/space complexity, 2D core array was used.



A 32x32 bit core memory plane

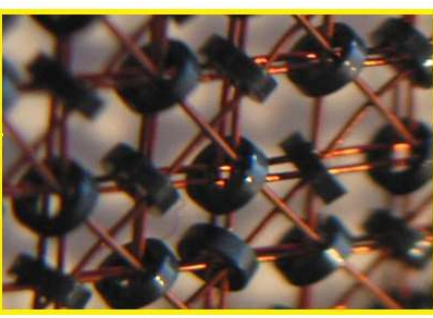
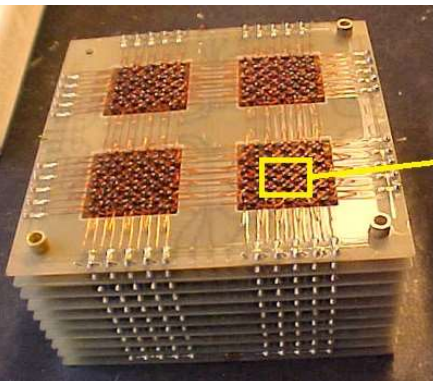
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

8

### Core Memory (3D arrangement)

Piling 2D planes, a 3D core array can be implemented.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

9

Static RAM (SRAM)

A 4x2 bits RAM. Each cell is a simple flip flop (2 controlled NOTs).

Switch

Control Line

Input

$I_1$

$I_0$

word 0

word 1

word 2

word 3

Chip select

Write

Address Lines

A0

A1

Address Decoder

Output

$O_1$

$O_0$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

10

Static RAM (SRAM)

A 4x3 bits SRAM. Each cell is a controlled SR flip flop.

Binary Cell logic

select

input

output

read/write

Binary Cell

select

input

output

read/write

2x4 Decoder

Address Lines

A0

A1

Memory Enable

read/write

Input Data

Word 0

Word 1

Word 2

Word 3

Output Data

A 4x3 bit SRAM

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

11

# Dynamic RAM (DRAM)

Each cell is a controlled capacitor.

wordline

+Vcc

Capacitor

bitline

'1' bit stored

wordline

0 v

Capacitor

bitline

'0' bit stored

From row decoder

From column decoder

Sense amplifier

Sense amplifier

Sense amplifier

Sense amplifier

A 16 bit DRAM

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

12

# Read Only Memory (ROM)

- Perform read operation only; write operation is not possible
- Information stored in a ROM is made permanent during production and cannot be changed
- Information on the data output line depends only on the information on the address input lines.

→ It is a combinational logic circuit

Example: A 4x4 bits ROM.

2x4 Decoder

A0

A1

Memory Enable

0

1

2

3

O<sub>0</sub>

O<sub>1</sub>

O<sub>2</sub>

O<sub>3</sub>

Address Lines

A0

A1

Memory Enable

0011

1110

0101

1010

O<sub>3</sub>O<sub>2</sub>O<sub>1</sub>O<sub>0</sub>

Output Data Lines

6

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

13

### Diode Implementation of ROMs

AND and OR gates can be implemented using diodes.

**Example:** Diode-based 4 input AND and OR gates.

The left diagram shows a 4-input AND gate. It consists of four diodes connected in series between inputs a, b, c, and d and a common output node. A pull-up resistor connects this node to +Vcc. The output is labeled a.b.c.d. The right diagram shows a 4-input OR gate. It consists of four diodes connected in parallel between inputs a, b, c, and d and a common output node. A pull-down resistor connects this node to GND. The output is labeled a+b+c+d.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

14

### Diode Implementation of ROMs

Any ROM can be implemented using an array of diodes.

**Example:** Design the following 8x4 bits ROM.

The diagram shows an 8x4 bit ROM implemented with diodes. On the left, a 3x8 decoder takes address lines A0, A1, and A2 as inputs and produces eight output lines labeled 0 through 7. Each output line is connected to a horizontal row of diodes. The vertical lines represent the four data outputs, O0, O1, O2, and O3, each with a pull-down resistor to GND. A Memory Enable signal is connected to the enable input E of the decoder. On the right, a table shows the 8x4 bit data stored in the ROM:

| Address Lines | O <sub>3</sub> | O <sub>2</sub> | O <sub>1</sub> | O <sub>0</sub> |
|---------------|----------------|----------------|----------------|----------------|
| 1010          | 1              | 0              | 1              | 0              |
| 0000          | 0              | 0              | 0              | 0              |
| 1011          | 1              | 0              | 1              | 1              |
| 1101          | 1              | 1              | 0              | 1              |
| 0001          | 0              | 0              | 0              | 1              |
| 0110          | 0              | 1              | 1              | 0              |
| 0110          | 0              | 1              | 1              | 0              |
| 1010          | 1              | 0              | 1              | 0              |

The output lines are labeled O<sub>3</sub>O<sub>2</sub>O<sub>1</sub>O<sub>0</sub> and the entire set of outputs is labeled Output Data Lines.

## Types of ROM

### ROM

- Store information (function) during production
- Mask is used in the production process
- Unalterable
- Low cost for large quantity production → used in the final products

### PROM (Programmable ROM)

- Store info electrically using PROM programmer at the user's site
- Unalterable
- Higher cost than ROM → used in the system development phase  
→ can be used in small quantity system

### EPROM (Erasable PROM)

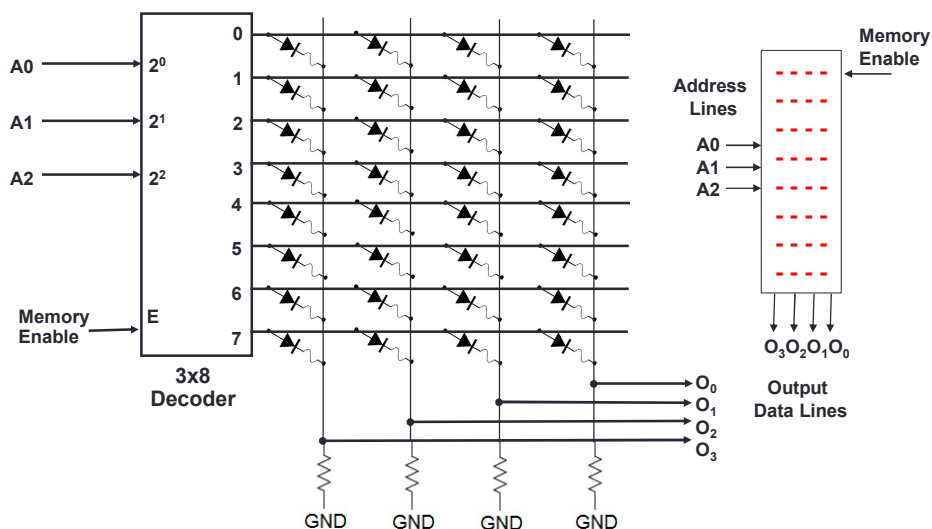
- Store info electrically using PROM programmer at the user's site
- Stored info is erasable (alterable) using UV light
- Higher cost than PROM but reusable → used in the system development phase, not in final production due to erasability

### E2PROM (Electrically EPROM)

- Store info electrically using PROM programmer at the user's site
- Stored info is electrically erasable
- Higher cost than EPROM but more durable → used in the system development phase, not in final production due to erasability

## Programmable ROM (PROM)

An empty ROM ready to be programmed (Example: 8x4 bits PROM).



## Erasable PROM (EPROM)

A PROM that can be erased when needed using ultra-violet light.

Fuses are of resistive type that can be in amorphous state (with very large resistance) and crystalline (almost no resistance). Under ultra-violet light for about 10-15 minutes, all resistors change to crystalline state.

When programming, selected resistors are electrically programmed to amorphous state.



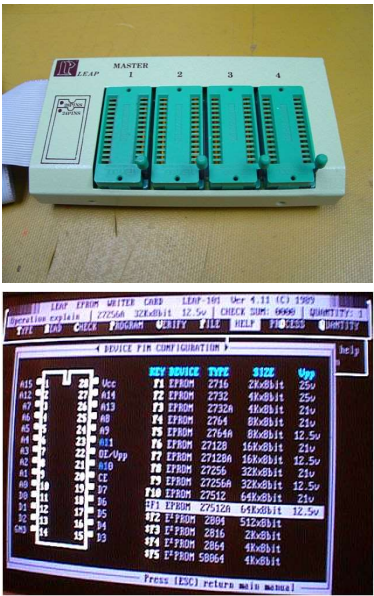
## Electrically EPROM (E<sup>2</sup>PROM)

A PROM that can be erased when needed using electrical current.

Fuses are of resistive type that can be in amorphous/crystalline state.

A special timing of an electrical current applied to the chip can be used to reset all resistors to crystalline state.

For programming, selected resistors are electrically programmed to amorphous state using some special timing with some specific current.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

19

### Memory Expansion

Widening the access width of the memory can be easily done by repeating memory modules of narrower access width.

**Example:** Design a 1024x16 RAM using two 1024x8 RAMs.

The diagram illustrates the expansion of memory access width. Two 1024x8 RAM modules, labeled '1024x8 RAM #0' and '1024x8 RAM #1', are connected to a common 1024x16 RAM. The Address Lines A0-A9 (10 bits) are connected to the Address input of both RAMs. The Data Lines D0-D15 (16 bits) are connected to the Data input/output of both RAMs. The RAMs are controlled by Read, Write, and Enable signals.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

20

### Memory Expansion

Expanding the address space of the memory, can be done using memory modules with smaller address space.

**Example:** Design a 4096x8 RAM using 1024x8 RAMs.

The diagram illustrates the expansion of memory address space using high-order interleaving. Four 1024x8 RAM modules, labeled 'RAM #0', 'RAM #1', 'RAM #2', and 'RAM #3', are connected to a common 4096x8 RAM. The Address Lines A0-A11 are used. A10 and A11 are used for a 2-to-4 decoder to enable one of the four RAMs. Address lines A0-A9 are connected to the Address input of each RAM. Data lines D0-D7 are connected to the Data input/output of each RAM. The RAMs are controlled by Read/Write signals.

**High-order Interleaving**

RAM #0: 0...1023, 1024x8  
RAM #1: 1024...2047, 1024x8  
RAM #2: 2048...3071, 1024x8  
RAM #3: 3072...4095, 1024x8

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

21

### Memory Expansion

Expanding the address space of the memory, can be done using memory modules with smaller address space.

**Example:** Design a 4096x8 RAM using 1024x8 RAMs.

**Low-order Interleaving**

The diagram illustrates the low-order interleaving of four 1024x8 RAM modules (RAM #0 to RAM #3) to create a 4096x8 RAM. The address lines A0-A11 are used, with A0-A1 connected to a 2-to-4 decoder. The decoder's outputs (0, 1, 2, 3) are connected to the enable (E) inputs of the RAM modules. The address lines A2-A11 are connected to the address inputs of the RAM modules. The data lines D0-D7 are connected to the data inputs/outputs of the RAM modules. The Read/Write control signal is connected to the R/W' inputs of the RAM modules.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

22

### Memory Expansion

**Example:** Design a 8Kx16 main memory (first 2K addresses as ROM, and next 6K as RAM) using 2Kx8 ROMs and 2Kx8 RAMs.

The diagram illustrates the design of a 8Kx16 main memory using eight 2048x8 modules. The address lines A0-A12 are used, with A0-A10 connected to a 2-to-4 decoder. The decoder's outputs (0, 1, 2, 3) are connected to the enable (E) inputs of the modules. The address lines A11 and A12 are connected to the address inputs of the modules. The data lines D0-D7 and D8-D15 are connected to the data inputs/outputs of the modules. The Read/Write control signal is connected to the RD and RD/WR' inputs of the modules.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

23

### Associative (content addressable) Memory

Unlike conventional memory, here we give data and search for the locations containing it or part of it.

A mxn bit associative memory

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

24

### Associative (content addressable) Memory

Internal Logic of one line of an mxn bit CAM.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

25

Associative (content addressable) Memory

Example: Search operation in an 8x6 bit CAM.

Argument Register

101110

Key Register

001101

Data Array

100001  
101011  
111100  
101010  
010111  
110011  
101110  
000000

Match Register

0  
0  
0  
0  
0  
0  
0  
0

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

26

Associative (content addressable) Memory

Example: Search operation in an 8x6 bit CAM.

Argument Register

101110

Key Register

001101

Data Array

100001  
101011  
111100  
101010  
010111  
110011  
101110  
000000

Match Register

0  
0  
0  
0  
0  
0  
0  
0

## Associative (content addressable) Memory

**Example:** Search operation in an 8x6 bit CAM.

Argument Register 101110

Key Register 001101

|            |        |   |
|------------|--------|---|
| Data Array | 100001 | 0 |
|            | 101011 | 0 |
|            | 111100 | 1 |
|            | 101010 | 0 |
|            | 010111 | 0 |
|            | 110011 | 0 |
|            | 101110 | 1 |
|            | 000000 | 0 |

Match Register

## Auxiliary (Secondary) Memory

The memory levels below the main memory level is known as secondary memory.

Popular auxiliary memory components:

- Disk storage
  - Hard disk
  - Floppy/zip disk
  - CD (compact disk)
  - SSD (solid state drive)
- Magnetic Tape storage
- Magnetic Drum storage

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

29

Disk Storage

rotating disk

moveable arm

read/write head

Single disk drive

sector

tracks

read/write head

Write operation

Read operation

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

30

Disk Storage

Each circle represents two tracks: one for top surface and one for bottom surface

Cylinder X made from Track X of all 12 surfaces (2 per platter)

Head Assembly  
All heads move together

Stack of disks

Spindle

Sector

Platter

Top and bottom Surfaces

Arm

Track

Head

Shaft

15

(c) Hamid Sarbazi-Azad

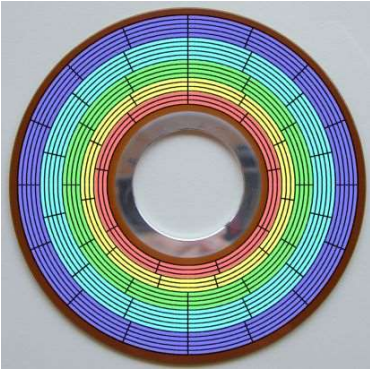
Computer Architecture -- Lecture #5: Memory system

31

# Disk Storage



Normal (Non zoned) recording



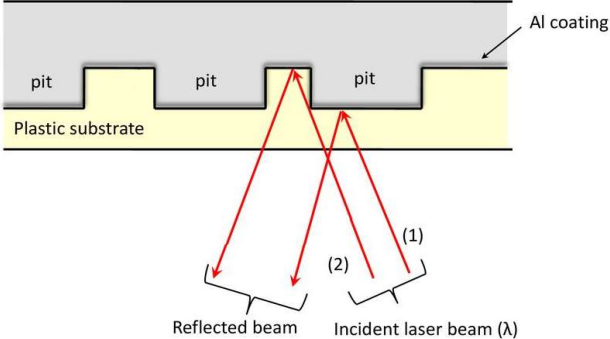
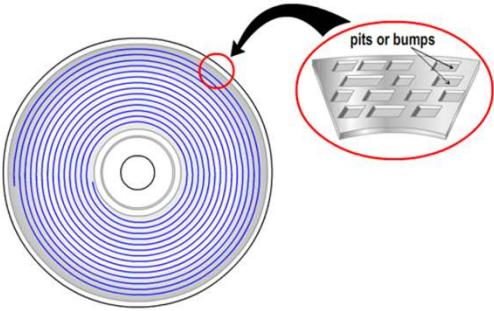
Zoned recording

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

32

# Disk Storage: CD



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

33

# Magnetic Tape Storage

end-of-file mark

end-of-record mark

Characters

parity bit

|     |     |     |   |      |       |
|-----|-----|-----|---|------|-------|
| 1 1 | 1 1 | 01  | 0 | 1001 | 01011 |
| 0 0 | 0 0 | 00  | 1 | 0000 | 00000 |
| 0 0 | 0 0 | 10  | 0 | 0010 | 00000 |
| 1 1 | 0 0 | ... | 0 | 1101 | ...   |
| 0 0 | 1 0 | 00  | 0 | 0010 | 00010 |
| 1 1 | 0 0 | 00  | 0 | 1001 | 11001 |
| 1 1 | 1 0 | 01  | 0 | 0010 | 00111 |
| 0 0 | 0 0 | 00  | 1 | 0000 | 01000 |
| 1 1 | 1 1 | 11  | 1 | 1111 | 11111 |

Record Gap Record

Frames

Tracks

Parity Track

take-up reel

supply reel

read/write head

drive rollers

Basic structure of tape drive

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

34

# Magnetic Tape Storage

The IBM 701 Tape Drive (1949)

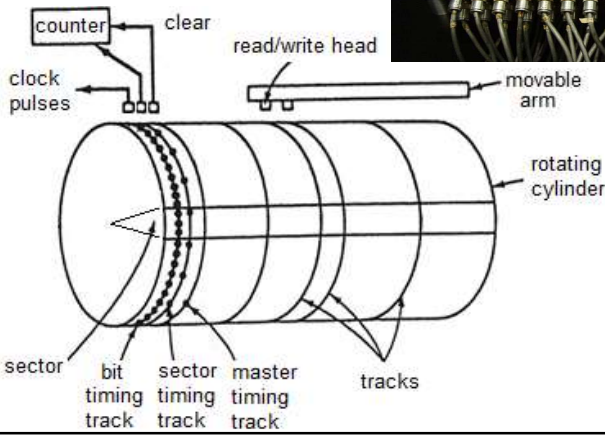
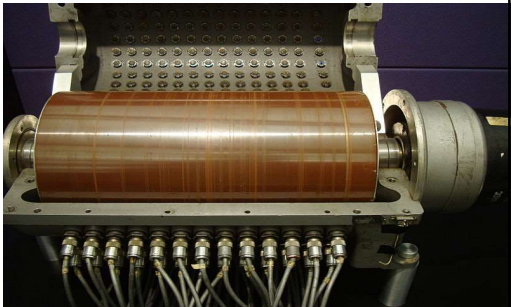
A modern robot-operated tape library (2010)

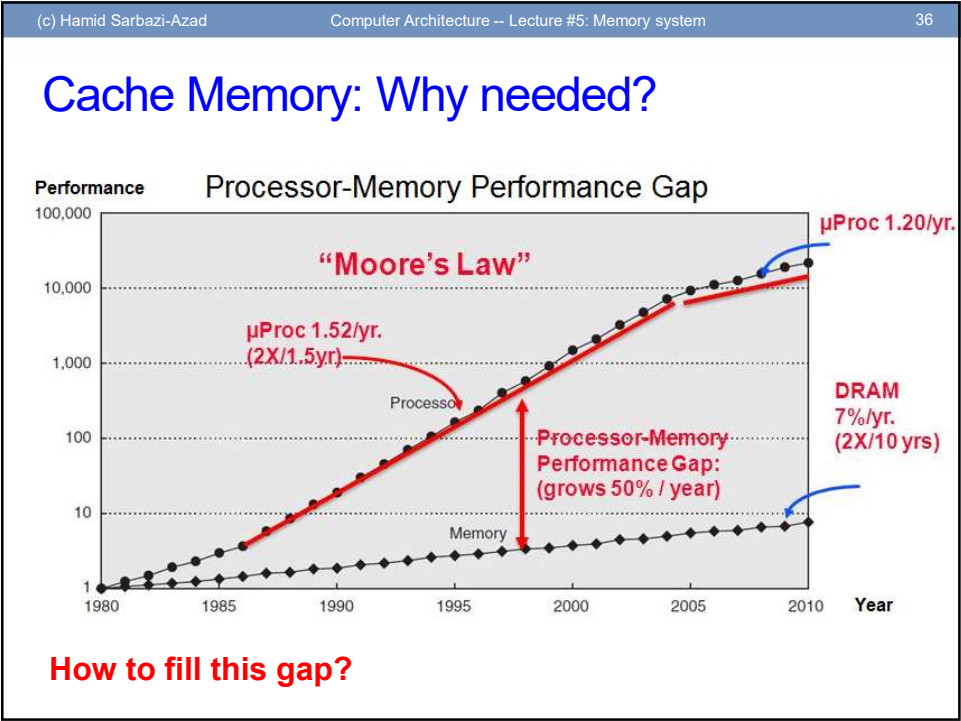
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

35

# Magnetic Drum





(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

37

# Cache Memory

Everything is a cache for something else in the hierarchy...

On the datapath

Registers

Level 1 Cache

Level 2 Cache

On chip

Level 3 Cache

Other chips

Main Memory

Flash Drive

Mechanical devices

Hard Disk

|               | Access time | Capacity | Managed By        |
|---------------|-------------|----------|-------------------|
| Registers     | 1 cycle     | 1 KB     | Software/Compiler |
| Level 1 Cache | 2-4 cycles  | 32 KB    | Hardware          |
| Level 2 Cache | 10 cycles   | 256 KB   | Hardware          |
| Level 3 Cache | 40 cycles   | 10 MB    | Hardware          |
| Main Memory   | 200 cycles  | 10 GB    | Software/OS       |
| Flash Drive   | 10-100us    | 100 GB   | Software/OS       |
| Hard Disk     | 10ms        | 1 TB     | Software/OS       |

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

38

# Cache Memory

Consider a computer with a cache memory between CPU and main memory as shown. Assume location X is accessed n times in a loop.

Average access time to location X is:

$$t_{avg} = \frac{t_M + n \times t_C}{n}$$
$$\lim_{n \rightarrow \infty} t_{avg} = t_C$$

CPU

C.M.

M.M.

$t_C$

$t_M$

→ if we frequently access a given location, the access is realized at almost cache memory speed!

## Cache Memory

It works due to the principle of **Locality of References (LoR)**.

Locality of references are either **Temporal** or **Spatial**.

**Temporal locality:** A location accessed will be very probably accessed again in near future.

**Example:** Accessing a variable in a loop.

**Spatial locality:** A location accessed is near by the location to be next accessed.

**Example:** Accessing elements of an array or instruction of a program.

**Example:** Show LoRs in the given code?

```
sum = 0;
for (i=0; i<100; i++)
    sum += a[i];
```

## Cache Memory: Definitions

On a generated address by CPU, cache memory is first accessed.

A **cache hit** occurs if the cache contains requested data. Hits are good as data can be given to CPU sooner.

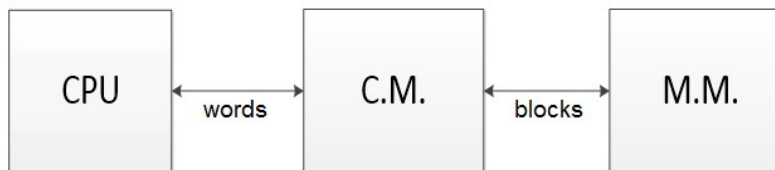
A **cache miss** occurs if the cache does **not** contain requested data. Misses are bad since we must access slow main memory to get requested data.

**Hit rate** (usually shown by **h**) is the percentage of memory accesses handled by the fast cache (SRAM).

**Miss rate** (**1-h**) is the percentage of memory accesses handled by the slower main memory (DRAM).

Typical caches have a high hit rate (95% or higher for I-cache 😊)

## Cache Memory



Cache and main memory are organized in **blocks** (or **lines**).

Block is the transfer unit between cache and main memory, while words are transferred between the cache and CPU.

The number of blocks in the cache and main memory is usually a power of 2.

When a requested address is missed in the cache, the block containing it is taken into the cache from main memory.

## Cache Memory

### Questions:

1. On miss, we bring a block of main memory to the cache. Where exactly should we put it in the cache?  
→ Mapping function (or placement algorithm)
2. How can we tell if a word is already in the cache, or if it must be fetched from main memory first?  
→ Valid bit
3. Eventually, the small cache memory might fill up. To load a new block from main memory, we must replace one of the existing blocks in the cache; which one?  
→ Replacement policy
4. How can WRITE operations be handled by the memory system?  
→ Write scheme

## Cache Memory

**Mapping function** indicates where the block resides in the cache.



Mapping functions:

- Direct mapping
- Associative mapping
- Set-associative mapping
- Sector mapping (not popular)

## Cache Memory: Direct Mapping

A deterministic hashing function is used to find the location in the cache to take the block from the main memory.

**Assume:**

A cache memory of  $2^b$  blocks each of  $2^w$  words.

→ Cache memory capacity =  $2^{b+w}$  words

Main memory has a capacity of  $2^c$  X the cache memory capacity.

→ Main memory capacity =  $2^{c+b}$  blocks =  $2^{c+b+w}$  words

Assume the addressable unit size is the word size.





(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

49

Cache Memory: Direct Mapping

What happens on a cache hit?

The CPU tries to read from memory. The address is sent to cache controller:

1. The Block filed (b bits) of address will index a block in the cache.

2. If the block is valid and its corresponding Tag matches the Tag filed of CPU address, then a hit occurs.

3. The Word field (w bits) indexes the block to find the requested word and is passed to CPU.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

50

Cache Memory: Direct Mapping

What happens on a cache hit?

Tag (block ID)

Block

Word

CPU Address

c bits

b bits

w bits

V

Tag

Data

B<sub>0</sub>

B<sub>1</sub>

⋮

B<sub>block</sub>

⋮

B<sub>2<sup>b</sup>-1</sub>

W<sub>0</sub>

W<sub>1</sub>

⋮

W<sub>word</sub>

⋮

W<sub>2<sup>w</sup>-1</sub>

①

→

B<sub>block</sub>

③

→

W<sub>word</sub>

→

To CPU

=

→

AND

→

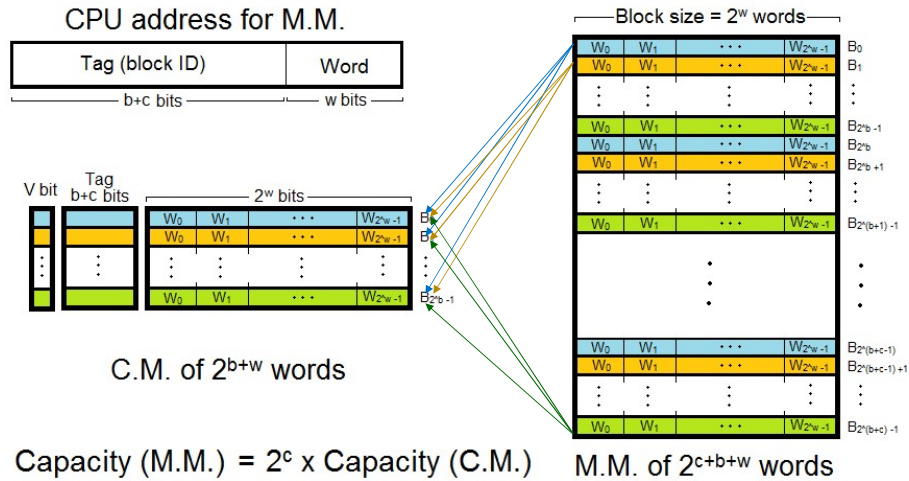
hit

②



## Cache Memory: Associative Mapping

The generated address by CPU has two fields:



## Cache Memory: Associative Mapping

## What happens on a cache hit?

The CPU tries to read from memory. The address is sent to cache controller:

1. The Tag filed ( $b+c$  bits) of address will be searched in the Tag array (like in associative memory).

→ This is time-consuming!!!

2. If found, the corresponding V bit is verified. If valid, then a hit occurs.

3. The Word field (w bits) is index on the corresponding block of the cache and the requested word is passed to CPU.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

55

### Cache Memory: Associative Mapping

What happens on a cache hit?

Tag (block ID)      Word      CPU Address

$b+c$  bits       $w$  bits

Cache structure:

|                              | V | Tag            | Data                                            |
|------------------------------|---|----------------|-------------------------------------------------|
| B <sub>0</sub>               |   | W <sub>0</sub> | W <sub>1</sub> ... W <sub>2<sup>w</sup>-1</sub> |
| B <sub>1</sub>               |   | W <sub>0</sub> | W <sub>1</sub> ... W <sub>2<sup>w</sup>-1</sub> |
| ...                          |   | ...            | ...                                             |
| B <sub>2<sup>b</sup>-1</sub> |   | W <sub>0</sub> | W <sub>1</sub> ... W <sub>2<sup>w</sup>-1</sub> |

① → Tag comparison      ② → hit      ③ → Data output

To CPU

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

56

### Cache Memory: Associative Mapping

**Example:**  
A computer has a main memory of  $2^{16}$  addressable bytes (word size = 2 bytes), and a fully-associative mapped cache of 32 words arranged in 8 blocks (each block of 4 words).

Show the internal structure and CPU address fields.

CPU Address for Main Memory

Tag (13 bits)      Word Byte

Cache structure:

|                | 13 bits                                                     | 64 bits                                                     |
|----------------|-------------------------------------------------------------|-------------------------------------------------------------|
| B <sub>0</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>1</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>2</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>4</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>5</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>6</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |
| B <sub>7</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> | W <sub>0</sub> W <sub>1</sub> W <sub>2</sub> W <sub>3</sub> |

V      Tag Array      Data Array

### Cache Memory: Set-Associative Mapping

In associative cache, keeping the Tag array is expensive and searching in it is time-consuming. Besides, having complete freedom to place a memory block in any cache block is not necessary.

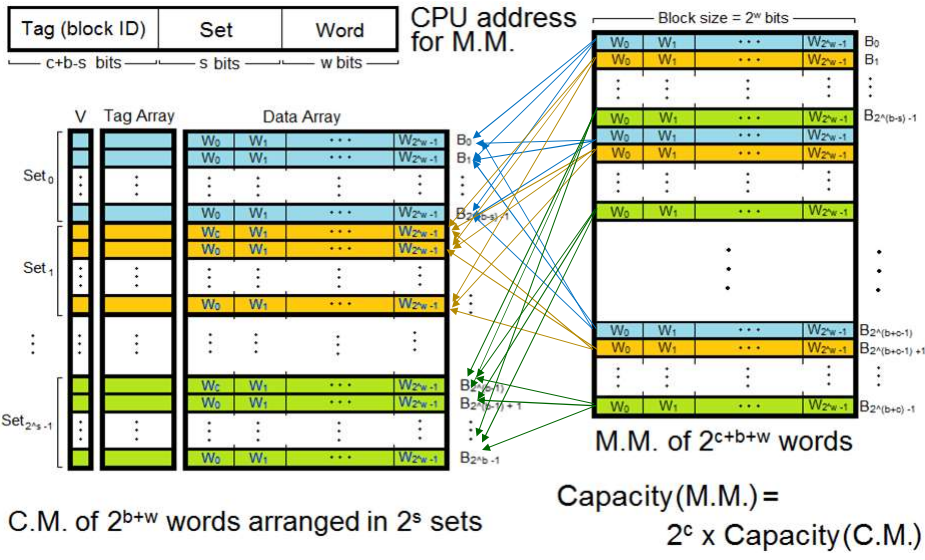
**Solution** → Set-Associative mapping function

In Set-Associative mapping, each main memory block is mapped to a specific (determined by a hashing function) set of blocks in the cache, but inside that set any cache block can host the memory block.

In an m-way Set-associative cache each set contains m blocks.

### Cache Memory: Set-Associative Mapping

The generated address by CPU has two fields:



(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #5: Memory system59

## Cache Memory: Set-Associative Mapping

What happens on a cache hit?

The CPU tries to read from memory. The address is sent to cache controller:

1. The Set filed (s bits) of address will index on cache sets.
2. The Tag field is searched in the Tag array of the set (a small associative memory). → quick search!
3. If found, the corresponding V bit is verified. If valid, then a hit occurs.
4. The Word field (w bits) is index on the corresponding block of the set and the requested word is passed to CPU.

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #5: Memory system60

## Cache Memory: Set-Associative Mapping

What happens on a cache hit?

The diagram illustrates the internal structure of a set-associative cache. At the top, a 'CPU Address' is divided into three fields: 'Tag (block ID)' (c+b-s bits), 'Set' (s bits), and 'Word' (w bits). The 'Set' field is used to index into a vertical array of 'Set' blocks, labeled from Set<sub>0</sub> to Set<sub>2<sup>s</sup>-1</sub>. Each 'Set' block contains a 'V' (Valid) bit and a 'Tag Array'. The 'Tag Array' is a small associative memory where the 'Tag' field of the CPU address is searched (step 2). If a match is found, the corresponding 'V' bit is checked (step 3). If 'V' is 1, it's a 'hit'. The 'Word' field of the CPU address is then used to index into the 'Data Array' of that set, which contains words W<sub>0</sub> to W<sub>2<sup>w</sup>-1</sub>. The requested word is then passed 'To CPU' (step 4). The sets are organized into blocks B<sub>0</sub> to B<sub>2<sup>s</sup>(b-1)-1</sub>.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

61

### Cache Memory: Set-Associative Mapping

**Example:**  
A computer has a main memory of  $2^{16}$  addressable bytes (word size = 4 bytes), and a 4-way set-associative mapped cache of 64 words arranged in 16 blocks (each block of 4 words).

Show the internal structure and CPU address fields.

CPU Address for Main Memory

10 bits      128 bits

V      Tag Array      Data Array

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

62

### Cache Memory: Mapping Function

Let us compare the relative standing of different mapping functions.

| Mapping Function          | Hardware Cost | Hit Latency | Hit Rate |
|---------------------------|---------------|-------------|----------|
| Direct Mapping            | ☺☺☺           | ☺☺☺         | ☺        |
| Set-Associative Mapping   | ☺☺            | ☺☺          | ☺☺       |
| Fully-Associative Mapping | ☹             | ☹           | ☺☺☺      |

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

63

### Cache Memory: Hit Rate Analysis

Hit rate is determined by

- Program characteristics
  - Temporal locality
  - Spatial locality
- Cache organization
  - Cache and block size
  - Associativity
  - Number of sets

hit rate

1

$1 - C^{-0.5}$

Capacity of the cache memory (C)

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

64

### Cache Memory: Hit Rate Analysis

Hit rate is determined by

- Program characteristics
  - Temporal locality
  - Spatial locality
- Cache organization
  - Cache and block size
  - Associativity
  - Number of sets

hit rate

1

optimal

Line size (fixed sized cache)

hit rate

1

fully associative

direct

Number of sets (fixed sized cache)

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

65

### Cache Memory: Replacement Policy

When the whole cache is full in a **fully-associative cache** or when a set is full in a **set-associative cache**, what happens if a miss occurs?

One valid block of the cache/set must be **replaced** (or evicted). The replaced (evicted) block is called **victim block**.

**Replacement policy** indicates which block must leave the cache. Some policies include:

- RND (Random) policy
- FIFO (First In First Out) policy
- LRU (Least Recently Used) policy
  - Simpler forms:
    - NRU (Not Recently Used) policy
    - Pseudo LRU policy
- LFU (Least Frequently Used) policy
- OPT (Optimal) policy
  - Impossible!! But shows opportunity 😊

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

66

### Cache Memory: Replacement Policy

For the block reference sequence:

1 → 7 → 4 → 5 → 1 → 7 → 6 → 5 → 2 → 4 → 3 → 2

in a 4-block associative cache with RND replacement policy, what is the hit and miss rate?

|              |   |   |   |   |   |   |   |   |   |   |   |   |
|--------------|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref. Seq.    | 1 | 7 | 4 | 5 | 1 | 7 | 6 | 5 | 2 | 4 | 3 | 2 |
| CACHE BLOCKS | - | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 3 | 3 |
|              | - | - | 7 | 7 | 7 | 7 | 7 | 7 | 2 | 4 | 4 | 2 |
|              | - | - | - | 4 | 4 | 4 | 4 | 5 | 5 | 5 | 5 | 5 |
|              | - | - | - | 5 | 5 | 5 | 6 | 6 | 6 | 6 | 6 | 6 |
|              | m | m | m | m | h | h | m | m | m | m | m | m |

Hit Rate (h) = 2/12 ≈ 0.17 → 17%

Miss Rate = 1 – h ≈ 0.83 → 83%

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

67

### Cache Memory: Replacement Policy

For the block reference sequence:

1 → 7 → 4 → 5 → 1 → 7 → 6 → 5 → 2 → 4 → 3 → 2

in a 4-block associative cache with FIFO replacement policy, what is the hit and miss rate?

|           |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref. Seq. |   | 1 | 7 | 4 | 5 | 1 | 7 | 6 | 5 | 2 | 4 | 3 | 2 |
| CACHE     | - | 1 | 1 | 1 | 1 | 1 | 1 | 6 | 6 | 6 | 6 | 6 | 6 |
| BLOCKS    | - | - | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 2 | 2 | 2 | 2 |
|           | - | - | - | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 3 | 3 |
|           | - | - | - | - | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
|           |   | m | m | m | m | h | h | m | h | m | h | m | h |

Hit Rate (h) = 5/12 ≈ 0.42 → 42%

Miss Rate = 1 – h ≈ 0.58 → 58%

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

68

### Cache Memory: Replacement Policy

For the block reference sequence:

1 → 7 → 4 → 5 → 1 → 7 → 6 → 5 → 2 → 4 → 3 → 2

in a 4-block associative cache with LRU replacement policy, what is the hit and miss rate?

|           |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref. Seq. |   | 1 | 7 | 4 | 5 | 1 | 7 | 6 | 5 | 2 | 4 | 3 | 2 |
| CACHE     | - | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 |
| BLOCKS    | - | - | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 4 | 4 | 4 |
|           | - | - | - | 4 | 4 | 4 | 4 | 6 | 6 | 6 | 6 | 3 | 3 |
|           | - | - | - | - | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
|           |   | m | m | m | m | h | h | m | h | m | m | m | h |

Hit Rate (h) = 4/12 ≈ 0.33 → 33%

Miss Rate = 1 – h ≈ 0.67 → 67%

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

69

### Cache Memory: Replacement Policy

For the block reference sequence:

1 → 7 → 4 → 5 → 1 → 7 → 6 → 5 → 2 → 4 → 3 → 2

in a 4-block associative cache with LFU replacement policy, what is the hit and miss rate?

|           |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref. Seq. |   | 1 | 7 | 4 | 5 | 1 | 7 | 6 | 5 | 2 | 4 | 3 | 2 |
| CACHE     | - | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| BLOCKS    | - | - | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 |
|           | - | - | - | 4 | 4 | 4 | 4 | 4 | 4 | 2 | 4 | 3 | 2 |
|           | - | - | - | - | 5 | 5 | 5 | 6 | 5 | 5 | 5 | 5 | 5 |
|           |   | m | m | m | m | h | h | m | m | m | m | m | m |

Hit Rate (h) = 2/12 ≈ 0.17 → 17%

Miss Rate = 1 – h ≈ 0.83 → 83%

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

70

### Cache Memory: Replacement Policy

For the block reference sequence:

1 → 7 → 4 → 5 → 1 → 7 → 6 → 5 → 2 → 4 → 3 → 2

in a 4-block associative cache with OPT replacement policy, what is the hit and miss rate?

|           |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref. Seq. |   | 1 | 7 | 4 | 5 | 1 | 7 | 6 | 5 | 2 | 4 | 3 | 2 |
| CACHE     | - | 1 | 1 | 1 | 1 | 1 | 1 | 6 | 6 | 6 | 6 | 6 | 6 |
| BLOCKS    | - | - | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 2 | 2 | 2 | 2 |
|           | - | - | - | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |
|           | - | - | - | - | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 3 | 3 |
|           |   | m | m | m | m | h | h | m | h | m | h | m | h |

Hit Rate (h) = 5/12 ≈ 0.42 → 42%

Miss Rate = 1 – h ≈ 0.58 → 58%



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

73

### Cache Memory: FIFO Anomaly

Now, consider same reference sequence and calculate hit rate for a 4-block cache with FIFO policy.

4 → 3 → 2 → 1 → 4 → 3 → 5 → 4 → 3 → 2 → 1 → 5

|           |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref. Seq. |   | 4 | 3 | 2 | 1 | 4 | 3 | 5 | 4 | 3 | 2 | 1 | 5 |
| 4 BLOCK   | - | 4 | 4 | 4 | 4 | 4 | 4 | 5 | 5 | 5 | 5 | 1 | 1 |
| CACHE     | - | - | 3 | 3 | 3 | 3 | 3 | 3 | 4 | 4 | 4 | 4 | 5 |
| (FIFO)    | - | - | - | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 3 | 3 | 3 |
|           | - | - | - | - | 1 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 |
|           |   | m | m | m | m | h | h | m | m | m | m | m | m |

Hit Rate (h) = 2/12 ≈ 0.17 → 17%

**The hit rate in a 3-block cache with FIFO policy was 25% !!!**

**→ FIFO Anomaly**

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

74

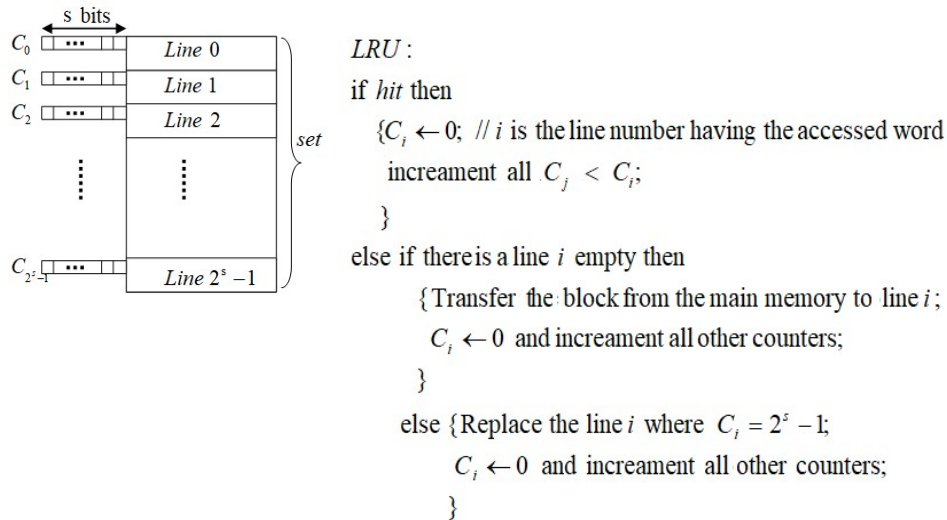
### Cache Memory: Stack Algorithm

In a **Stack algorithm** or policy, if we increase the size of cache (number of blocks), the hit ratio will **not be reduced**.

- FIFO is not a stack policy.
- It can be proved that LRU is a stack algorithm.

Any policy that does not take into account the reuse history is not a stack algorithm.

## Cache Memory: True LRU Implementation



## Cache Memory: NRU and Pseudo LRU Policies

As implementation of True LRU is costly, lighter versions of such schemes are designed.

**Not Recently Used (NRU)** is one of them. In this policy, each block of cache is associated with a bit called Recently Used (RU). Whenever a block is accessed, this bit is set.

On eviction, blocks with RU bit = 1 are kept and one of the block with RU=0 is replaced.

→ In a 2-way set-associative mapping, LRU and NRU are equivalent.

Other pseudo LRU policies were also implemented in the real processors.

For example, see the pseudo LRU technique used by Intel 80486 processor @ <https://en.wikipedia.org/wiki/Pseudo-LRU>



## Cache Memory: Write Schemes

**Write-through scheme:** When a word in a cache block is updated, it is also updated in main memory.

It is cool from consistency point of view, but is bad for performance as it increases traffic on main memory.

Average access time in a memory system with one level cache can be calculated as:

$$t_a = t_c + (1 - h)t_b + w(t_m - t_c)$$

where  $t_c$  is the access time to cache,  $t_b$  is the access time to a main memory block,  $h$  is the cache hit rate,  $w$  is the write rate, and  $t_m$  is the word access time in main memory.

## Cache Memory: Write Schemes

**Write-back scheme:** Let the cache block be updated while in the cache. On eviction, the block is written back to main memory.

It is cool from performance (no traffic overhead for main memory) point of view, but is bad for consistency of data between cache memory, main memory and I/O devices.

To implement this scheme we need another bit associated to each cache block called **Dirty** bit. When any word in a cache block is updated, we set its Dirty bit to 1. On eviction, if Dirty bit = 1, then the cache block is written back to main memory.

Average access time can be calculated as:

$$t_a = t_c + (1 - h)(t_b + dt_b)$$

where  $d$  is the probability that the block is dirty.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

81

### Cache Memory: Example

In a 2-address machine:

- Main memory size: 64 words (each 8 bits)
- A direct-mapped cache of 16 words (block size 8 words)
- Four general purpose registers: R0, R1, R2, R3
- Numbers are represented in 2s-complement
- Addressing modes: Direct, Immediate
- Two instruction formats

Instruction set includes:

| Opcode | Mnemonic    | Operation                                      |
|--------|-------------|------------------------------------------------|
| 01     | jmp address | $PC \leftarrow \text{Address};$                |
| 10     | jnz address | if (R0)≠0 then $PC \leftarrow \text{Address};$ |
| 11     | jz address  | if (R0)=0 then $PC \leftarrow \text{Address}$  |
| 00 0   | add r1,data | $r1 \leftarrow (r1) + \text{data}; \quad t=1$  |
| 00 0   | add r1,r2   | $r1 \leftarrow (r1) + (r2); \quad t=0$         |
| 00 1   | sub r1,data | $r1 \leftarrow (r1) - \text{data}; \quad t=1$  |
| 00 1   | sub r1,r2   | $r1 \leftarrow (r1) - (r2); \quad t=0$         |

Format I

|        |         |
|--------|---------|
| Opcode | Address |
| 2 bits | 6 bits  |

Format II

|        |        |        |
|--------|--------|--------|
| Opcode | r1     | Opr2   |
| 3 bits | 2 bits | 3 bits |

Opr2 = txx  
if t=0 then xx indicates a register  
if t=1 then xx is an immediate data

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #5: Memory system

82

### Cache Memory: Example

Q1. Determine the length of machine registers.  
Q2. Show the block diagram of the cache and CPU address fields.  
Q3. Determine the hit rate for program:  
20h, 26h, 0, 0, 0, 5Eh, 2Eh, 28h, BFh, BFh, 25h, 99h, 0  
started at address 20 (14h) in the main memory.

Answer 1:  $L_{MAR} = L_{PC} = 6 \text{ bits}$      $L_{MBR} = 8 \text{ bits}$      $L_{IR} = 8 \text{ bits}$      $L_{R0..R3} = 8 \text{ bits}$

Answer 2:

CPU address for M.M.

|        |       |        |
|--------|-------|--------|
| Tag    | Block | Word   |
| 2 bits | 1 bit | 3 bits |

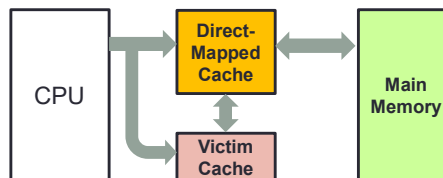
V Tag                      Data

|                |  |  |  |                |                |                |                |                |                |                |                |
|----------------|--|--|--|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| B <sub>0</sub> |  |  |  | W <sub>0</sub> | W <sub>1</sub> | W <sub>2</sub> | W <sub>3</sub> | W <sub>4</sub> | W <sub>5</sub> | W <sub>6</sub> | W <sub>7</sub> |
| B <sub>1</sub> |  |  |  | W <sub>0</sub> | W <sub>1</sub> | W <sub>2</sub> | W <sub>3</sub> | W <sub>4</sub> | W <sub>5</sub> | W <sub>6</sub> | W <sub>7</sub> |

## Cache Memory: Victim Cache

There are situations where a block recently evicted from cache memory (specifically in direct-mapped caches) will be soon missed.

To keep such useful blocks, a small fully-associative cache memory called Victim cache (4-16 block fully-associative) is used to keep evicted blocks until we make sure they are not useful.



### Operation of Victim cache:

1. If address hits in the cache do as usual.
2. If address hits in Victim cache, data is delivered to CPU and the hit block is swapped with corresponding block in cache.
3. If miss, take the block from main memory to cache and do as usual. If the host block is valid in cache, move it to Victim cache.

## END OF SLIDES

## QUESTIONS?